RUBY 101

Crash Course

Information Technology Institute

COURSE MATERIALS

You can access the course materials via this link

http://goo.gl/RLe4Gx

CONTENTS

- History
- Philosophy
- Features / Principles
- Installation
- Ruby Programs & Docs
- Syntax highlighting
- Control Structures
- Array & Hashes
- Symbols
- Blocks & Iterators
- OOP in Ruby

HISTORY

1993
Developed in Japan by Yukihiro Matsumoto (aka Matz)

1995
First public alpha version, gaining some popularity in Japan

1996
Core development team formed, Ruby 1.0 released

1998
First stable version released (1.2)

2003
Ruby 1.8 released, increasing popularity outside Japan

2005
David Heinemeier Hansson (aka DHH) released Ruby on Rails

2013
Ruby 2.0 released

2019
Ruby 2.7 released

> **The goal of Ruby is to make programmers happy. I started out to make a programming language that would make me happy, and as a side effect it's made many, many programmers happy!**
>
> **Matz**

Features

- Cross-platform
- General purpose
- Dynamic
- Interpreted
- Open source
- Free format
- Optional statement delimiters
- Expressive
- Object Oriented

Principles

- TIMTOWTDI
- Truly Object Oriented
- Principle of least astonishment (POLA)/surprise
- Duck typing
- Syntactic sugar
- Open Classes
- Metaprogramming
- Object messages

TIMTOWTDI

- In developing Ruby, Matz was heavily influenced by the programming languages he was already familiar with. **Larry Wall**, the developer of the popular Perl language, was a hero of Matz's, and Perl's principle of **there is more than one way to do it** (TIMTOWTDI) is present in Ruby.
- In contrast to Python which prefers to provide more rigid structures and present a small number of options to perform a certain task. Ruby gives a lot of ways to accomplish any task
- As example, the following are equivalents:

```
[4,5,8].inject { |sum, n| sum + n } → 17

[4,5,8].reduce(:+) → 17

[4,5,8].sum → 17
```

Truly Object Oriented

- Ruby is a genuine object-oriented language. Everything you manipulate is an object, even literal values

```
"gin joint".length → 9

"Rick".index("c") → 2

-1942.abs → 1942

'string'.class → String

'string'.class.class → Class

'string'.class.class.ancestors → [Class,
Module, Object, Kernel, BasicObject]

"Hello".method(:class).class → Method
```

POLA

- Principle of least astonishment (POLA), also called the principle of least surprise, applies to user interface and software design and states that "**If a necessary feature has a high astonishment factor, it may be necessary to redesign the feature**."

- After you learn Ruby well, you should be able to understand the logic behind everything, so you can predict the way it will work on other new features.

Duck typing

- Ruby objects (unlike objects in some other object-oriented languages) can be individually modified.
- We rely less on the type (or class) of an object and more on its capabilities. Hence, Duck Typing means an object type is defined by what it can do, not by what it is.
- If an object walks like a duck and quacks like a duck, then the Ruby interpreter is happy to treat it as if it were a duck.

```
puts 5

5.respond_to?(:to_s)
```

Syntactic sugar

- syntactic sugar to refer to special rules that let you write your code in a way that doesn't correspond to the normal rules but that is easier to remember how to do and looks better.

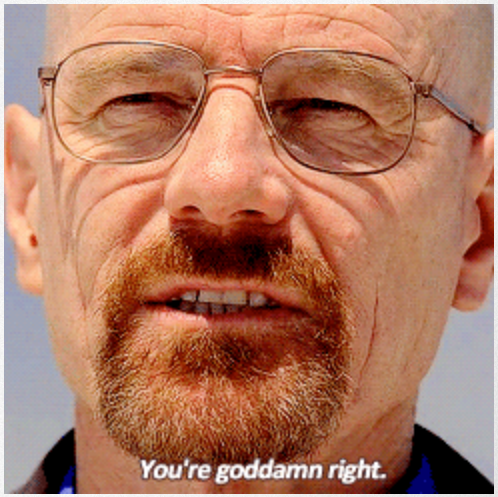
```
5.+(5)

5 + 5
```

Open Classes

- In Ruby, classes are never closed: you can always add methods to an existing class.

```
class String
  def say_my_name
    'Islam'
  end
end
```



Metaprogramming

- Metaprogramming is a technique by which you can write code that *writes* code by itself dynamically at runtime. This means you can define methods and classes during runtime!

```
class Developer
   def initialize(*args)
      args.each do |mtd|
         self.class.define_method "coding_#{mtd}"
do
            p "coding " + mtd.to_s
         end
      end
   end
end
```

Object messages

- Messages are sent in Ruby anytime a method is called on an object.

```
5.+(5)

5.send(:+, 5)
```

Installation

- Ruby can be installed by several ways, like:
  - System pacakge manager (Unix-like operating systems)
  - Ruby Installer (MS Windows)
  - Ruby Managers (RVM, rbenv or chruby)

- We *recommend* the installtion via **ruby managers**, as it gives more options and allows the installtion of more than one version of ruby on the same machine.
- For other options of installtions, refer to the official guide here:

https://www.ruby-lang.org/en/documentation/installation

Linux

- To check whether ruby is installed or not, open the **Terminal** and run:

```
ruby -v
```

- If Ruby is installed, it will give an output such as the following:

```
ruby 2.6.5p114 (2019-10-01 revision 67812)
```

- If not installed, you can install it via your distro package manger
  - Debian based (Debian, Ubuntu, …)

```
$ sudo apt-get install ruby-full
```

  - Redhat based (RHEL, CentOS, Fedora,…)

```
$ sudo yum install ruby
```

macOS

- By default ruby comes installed with MacOs, to check your version, open the **Terminal** and run:

```
ruby -v
```

- If Ruby is installed, it will give an output such as the following:

```
ruby 2.6.5p114 (2019-10-01 revision 67812)
[x86_64-darwin19]
```

- If not installed you can install it via **Homebrew**

```
brew install ruby
```

Windows

- Download Ruby Installer from this link RubyInstaller,
- After installation, open PowerShell or CMD and run:

```
C:/Ruby24-x64/bin/ridk.cmd
```

You should see the help options for this command:

```
Option:
    install     Install MSYS2 and MINGW dev tools
    exec        <command> Execute a command within MSYS2 context
    enable      Set environment variables for MSYS2
    disable     Unset environment variables for MSYS2
    version     Print RubyInstaller and MSYS2 versions
    use         Switch to a different ruby version
    help | --help | -? | /? Display this help and exit
```

RVM

- Ruby Version Manager (RVM) allows you to install and manage multiple installations of Ruby on your system. It can also manage different gemsets. It is available for macOS, Linux, or other UNIX-like operating systems.
- To install RVM with ruby run the follwoing:

```
$ gpg2 --recv-keys
409B6B1796C275462A1703113804BB82D39DC0E3
7D2BAF1CF37B13E2069D6956105BD0E739499BDB

$ \curl -sSL https://get.rvm.io | bash -s stable

$ rvm install 2.6.1
```

- For troubleshooting and other installation options refer to:

https://rvm.io/rvm/install

First program

- Open your editor and type the following:

```
puts 'Hello, world!'
```

- Save the file as **program.rb** to the documents directory, then open **Terminal** and type:

```
ruby ~/Documents/program.rb
```

- The result should be printed to the terminal!

```
Hello, world!
```

Documentation

- You can download HTML version of the documentation via http://www.ruby-doc.org
- You can use **ri** tool (ruby index) to browse documentation locally

```
ri Array
ri Array.sort
ri Hash#each
```

- You can also use offline documentation browsers, like Dash, Velocity and Zeal

irb

- To play with ruby locally, it‘s recommended to use **irb** (interactive ruby shell) to test and evaluate ruby statements and experssions.
- Open **Termianl** and type:

```
irb
```

```
irb(main):001:0>
```

- If it’s not installed, install it by typing:

```
gem install irb
```

Comments

```
# This is a comment


=begin

This is a multi-line comment.

The beginning line must start with "=begin"
and the ending line must start with "=end".
You can do this, or start each line in a
multi-line comment with the # character.

=end
```

Arithmetic Operators

```
1 + 1 #=> 2

8 - 1 #=> 7

10 * 2 #=> 20

35 / 5 #=> 7

2 ** 5 #=> 32

5 % 3 #=> 2
```

Syntax highlighting

Equality & Inequality

```
1 == 1 #=> true

2 == 1 #=> false


1 != 1 #=> false

2 != 1 #=> true
```

Syntax highlighting

Comparisons

```
1 < 10 #=> true

1 > 10 #=> false

2 <= 2 #=> true

2 >= 2 #=> true
```

Syntax highlighting

Made with ❤ by 

Combined comparison

```
# Combined comparison operator (returns `1` when the first argument is greater, # `-1` when the second argument is greater, and `0` otherwise)

1 <=> 10 #=> -1 (1 < 10)

10 <=> 1 #=> 1 (10 > 1)

1 <=> 1 #=> 0 (1 == 1)
```

Syntax highlighting

Falsey values

```
# Apart from false itself, nil is the only
other 'falsey' value

!!nil #=> false

!!false #=> false

!!0 #=> true

!!"" #=> true

!![] #=> true
```

Syntax highlighting

Strings

```
# Can use " or ' for Strings, but ' is more efficient
puts 'Hello, World!'
puts "Hello, World!"
```

Syntax highlighting

Strings

```
placeholder = 'use string interpolation'

"I can #{placeholder} when using double quoted strings"


# You can combine strings using `+`, but not with other types

'hello ' + 'world' #=> "hello world"

'hello ' + 3 #=> TypeError: can't convert Fixnum into String

'hello ' + 3.to_s #=> "hello 3"

"hello #{3}" #=> "hello 3"
```

Syntax highlighting

Strings

```
# ...or combine strings and operators

'hello ' * 3 #=> "hello hello hello "

# ...or append to string (Strings are
mutable)

'hello' << ' world' #=> "hello world"

# You can print to the output with a newline
at the end

puts "I'm printing!" #=> I'm printing! #=>
nil

# ...or print to the output without a newline
print "I'm printing!" #=> "I'm printing!" =>
nil
```

Syntax highlighting

Some useful string methods

```
a.concat(33)    #=> "hello world!"
"cat o' 9 tails" =~ /\d/    #=> 7
a[1]                        #=> "e"
a[2, 3]                     #=> "llo"
a[2..3]                     #=> "ll"
"hello".capitalize     #=> "Hello"
a.capitalize!    #=> "Hello"
"hello\n".chomp             #=> "hello"
```

Syntax highlighting

Some useful string methods

```
"hello".include? "lo"    #=> true

"hello".index('e')                 #=> 1

"abcd".insert(0, 'X')     #=> "Xabcd"


#Returns a printable version of str,
surrounded by quote marks, with special
characters escaped.

str = "hello"

str[3] = "\b"

str.inspect          #=> "\"hel\\bo\""
```

Syntax highlighting

Numbers

```
#In Ruby, numbers without decimal points are called integers, and numbers with decimal points are usually called floating-point numbers or, more simply, floats.

puts 1 + 2

puts 2 * 3

# Integer division

# When you do arithmetic with integers, you'll get integer answers

puts 3 / 2

puts 10 - 11

puts 1.5 / 2.6
```

Syntax highlighting

Numbers

```
# The increment and decrement operators (++
and --) are not available in Ruby, neither in
"pre" nor "post" forms. However, do note that
the += and -= are available

i+=1

i+-1


#Underscores may be inserted into integer
literals (though not at the beginning or
end), and this feature is sometimes used as a
thousands separator

1_000_000_000
```

Syntax highlighting

Variables

```
x = 25 #=> 25

x #=> 25

# Note that assignment returns the value
assigned.

# This means you can do multiple assignment.
x = y = 10 #=> 10


# By convention, use snake_case for variable
names. snake_case = true
```

Syntax highlighting

Made with ❤ by

Variables

```
#x, y = y, x will interchange the values of x and y.
X = 5 #=> 5
Y = 6 #=> 6


x, y = y, x


X #=> 6
Y #=> 5
```

Syntax highlighting

Variables

```
# Heredoc style is available
a = <<END_STR
This is the string
And a second line
END_STR
puts a
```

Syntax highlighting

Variables types

Variables				Constants and
Local	Global	Instance	Class	Class Names
name	$debug	@name	@@total	PI
fish_and_chips	$CUSTOMER	@point_1	@@symtab	FeetPerMile
x_axis	$_	@X	@@N	String
thx1138	$plan9	@_	@@x_pos	MyClass
_26	$Global	@plan9	@@SINGLE	JazzSong

Syntax highlighting

Variables

```
# Heredoc style is available
a = <<END_STR
This is the string
And a second line
END_STR
puts a
```

Syntax highlighting

NIL

```
# nil is an actual object. You can call
methods on nil, just like any other object.
You can add methods to nil, just like any
other object.

puts NIL.class # NilClass
```

Syntax highlighting

Made with ❤ by 

Console input

Write a script to get the user input in console and print it again to the user

Method definition

```
# method to great us before sleeping!
def say_goodnight(name)
   "Good night, " + name
end
# Time for bed...
puts say_goodnight(“Islam")
Or
puts say_goodnight “Islam”
```

Syntax highlighting

Method alias

```
def oldmtd
   "old method"
end

alias newmtd oldmtd
def oldmtd
   "old improved method"
end

puts oldmtd
puts newmtd
```

Syntax highlighting

Pass by Value vs Reference

```
def downer(string)
   string.downcase
end
a = "HELLO"
downer(a)         # -> "hello"
puts a            # -> "HELLO"

class << a
def downer
   self.replace downcase
end
end
```

Syntax highlighting

Made with ❤ by 

Bang (!) Methods

```
#Ruby methods that modify an object in-place
and end in an exclamation mark (!) are known
as bang methods. Also called dangerous
methods!


a = 'ISLAM'
a.downcase
a.downcase!
```

Syntax highlighting

Boolean (?) Methods

```
# Any method whose name ends with ? returns a
Boolean value


puts “Islam”.include?”s”
puts “Islam”.include?”z”
```

Syntax highlighting

Conditionals

```
if true
   'if statement‘
elsif false
   'else if, optional‘
else
   'else, also optional‘
end
x = 5
p 5 if x == 5
p '! 10‘ unless x == 10'
```

While

```
while weight < 100 and num_pallets <= 30
pallet = next_pallet()
weight += pallet.weight
num_pallets += 1
end

square = 2
square = square*square while square < 1000
```

Ternary Operator

```
(condition) ? (result if condition is true) :
(result if condition is false)

age = 15

# will output teenager

puts (13...19).include?(age) ? "teenager" :
"not a teenager"
```

Case

```
year = 2000

leap = case
       when year % 400 == 0 then true
       when year % 100 == 0 then false
       else year % 4   == 0
       end

puts leap

# output is: true
```

Symbol

```
# Symbols are immutable, reusable constants
represented internally by an integer value.


:pending.class #=> Symbol

status = :pending


# Strings can be converted into symbols and
vice versa.

status.to_s #=> "pending"

"argon".to_sym #=> :argon
```

Symbol vs String

```
puts "string".object_id
puts "string".object_id
puts :symbol.object_id
puts :symbol.object_id
```

Range

```
# Sequences have a start point, an end point, and
a way to produce successive values in the
sequence

(1..10).class ->Range

(1..10).to_a -> [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

(1...10).to_a -> [1, 2, 3, 4, 5, 6, 7, 8, 9]


digits = -1..8

puts digits.include?(5)        # true

puts digits.min                  # -1

puts digits.max                   # 8

puts digits.reject {|i| i < 5 }  # [5, 6, 7, 8]
```

Array

```
# This is an array.
array = [1, 2, 3, 4, 5] #=> [1, 2, 3, 4, 5]


# Arrays can contain different types of
items.
[1, 'hello', false] #=> [1, "hello", false]


# You might prefer %w instead of quotes
%w[foo bar baz] #=> ["foo", "bar", "baz"]
```

Array

```
# Arrays can be indexed.

# From the front...

array[0] #=> 1

array.first #=> 1

array[12] #=> nil

# ...or from the back...

array[-1] #=> 5

array.last #=> 5

# ...or with a start index and length...
array[2, 3] #=> [3, 4, 5]
```

Array

```
# ...or with a range...

array[1..3] #=> [2, 3, 4]

# Like arithmetic, [var] access is just
syntactic sugar for calling a method '[]' on
an object.

array.[] 0 #=> 1

array.[] 12 #=> nil

# You can add to an array...

array << 6 #=> [1, 2, 3, 4, 5, 6]

# Or like this

array.push(6) #=> [1, 2, 3, 4, 5, 6]
```

Array methods

```
newarr = [45, 23, 1, 90]
puts newarr.sort
puts newarr.length
puts newarr.include?(23)

# Parallel Assignment
a = 1, 2, 3 # => a == [1, 2, 3]
b = [1, 2, 3] # => b == [1, 2, 3]
a, b = 1, 2, 3 # => a == 1, b == 2
c, = 1, 2, 3, 4 # => c == 1
```

Hash

```
# Hashes are Ruby's primary dictionary with
  key/value pairs.
# Hashes are denoted with curly braces.
hash = { 'color' => 'green', 'number' => 5 }
hash.keys #=> ['color', 'number']
# Hashes can be quickly looked up by key.
hash['color'] #=> "green"
hash['number'] #=> 5
# Asking a hash for a key that doesn't exist
  returns nil.
hash['nothing here'] #=> nil
```

Hash

```
# When using symbols for keys in a hash, you
  can use an alternate syntax.
hash = { :defcon => 3, :action => true }
hash.keys #=> [:defcon, :action]

hash = { defcon: 3, action: true }
hash.keys #=> [:defcon, :action]

# Check existence of keys and values in hash
hash.key?(:defcon) #=> true
hash.value?(3) #=> true
```

Code block

```
# are chunks of code between braces or
  between do- end.
# can associate with method invocations,
  almost as if they were parameters
# Ruby standard is to use braces for single-
  line blocks and do- end for multi-line
  blocks.
x = 10
5.times do |x|
  puts "x inside the block: #{x}"
end
puts "x outside the block: #{x}"   #10
```

Code block

```
# pass code block to function to excute it
def call_block
puts "Start of method"
yield
yield
puts "End of method"
end
call_block { puts "In the block" }
```

Code block

```
# pass parameter to the code block

def call_block

yield("hello", 99)

end

call_block {|str, num| print str , " your
  number " , num }
```

Iterators

```
# methods that return successive elements
  from some kind of collection, such as an
  array.

# work mainly with blocks

animals = %w( ant bee cat dog elk )

animals.each {|animal| puts animal }


#for ... in

for i in ['fee', 'fi', 'fo', 'fum']

print i, " "

end
```

Iterators

```
[ 'cat', 'dog', 'horse' ].each {|name| print
  name, " " }

5.times { print "*"}

3.upto(6) {|i| print i }

6.downto(3) {|i| print i }

('a'..'e').each {|char| print char }
```

Exception handling

```
begin
# Code here that might raise an exception
  raise NoMemoryError, 'You ran out of memory.'
rescue NoMemoryError => exception_variable
  puts 'NoMemoryError was raised', exception_variable
rescue RuntimeError => other_exception_variable
  puts 'RuntimeError was raised now'
else
  puts 'This runs if no exceptions were thrown at all'
ensure
  puts 'This code always runs no matter what'
end
```

Read/Write Text Files

```
# Open and read from a text file
# Note that since a block is given, file will
# automatically be closed when the block terminates
File.open(‘test.rb', 'r') do |f1|
  while line = f1.gets
    puts line
  end
end

# Create a new file and write to it
File.open('test.rb', 'w') do |f2|
  # use "\n" for two lines of text
  f2.puts "Created by Islam\nThank God!"
end
```

Regular Expressions

```
/[0-9]/.class     # Regexp


m1 = /Ruby/.match("The future is Ruby")
puts m1.class  # it returns MatchData
m2 = "The future is Ruby" =~ /Ruby/
puts m2                 # it returns 14
```

regex

Including Other Files

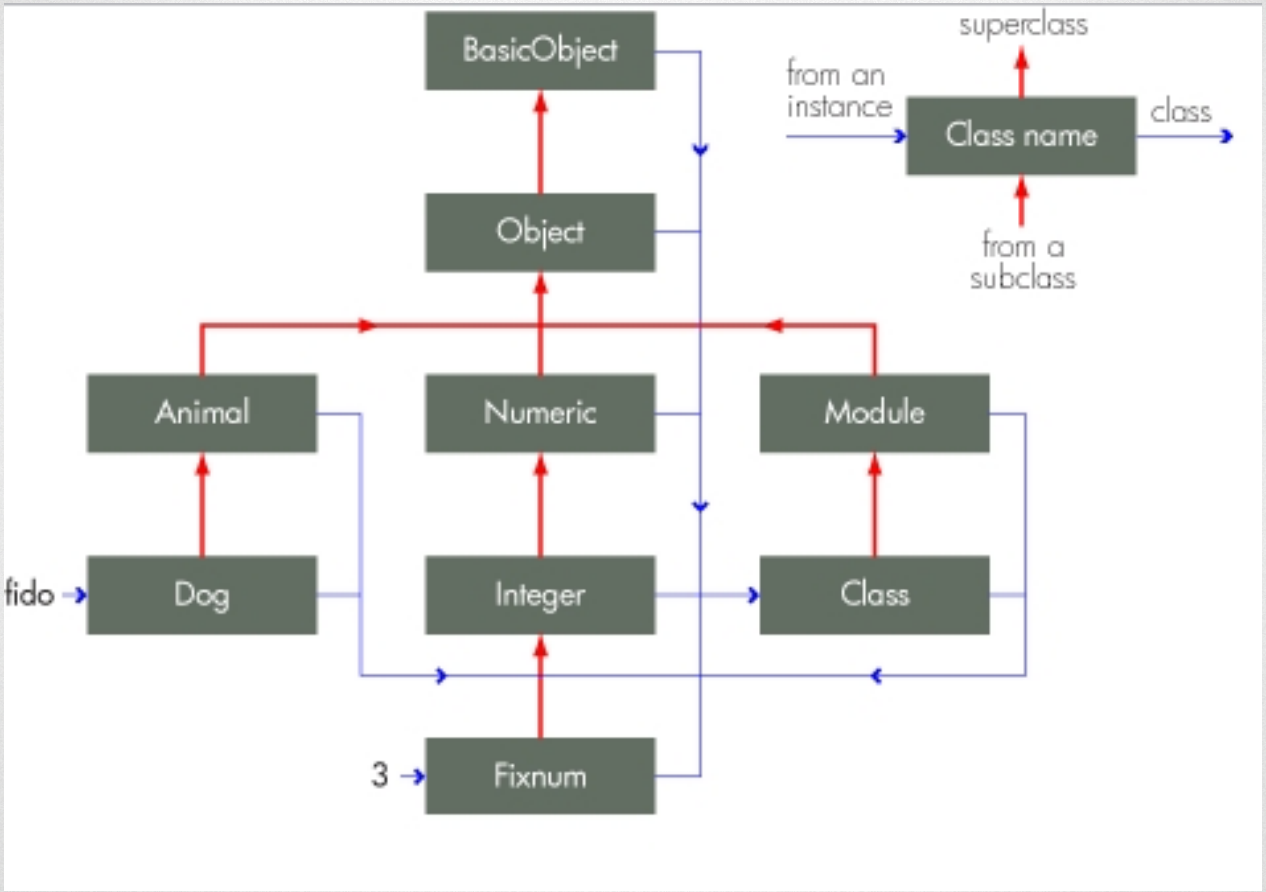
```
# The load method includes the named Ruby
  source file every time the method is
  executed:

load 'filename.rb'

# The more commonly used require method loads
  any given file only once:

require 'filename'
```

Object Orientation



Class definition

```
class Dog
  def initialize(name)
    # Instance variable
    @name = name
  end

  def bark
    puts 'Ruff! Ruff!'
  end
end
Dog.new(‘scary’)
```

Inheritance

```
class Person
    def initialize(name, age, job)
      @name = name
      @age= age
      @job = job
    end
end
class Student < Person
    def initialize(name, age, job, branch)
        super(name, age, job)
        @branch = branch
    end
end
```

Redefining Methods

```
class OR
  def mtd
    puts "First definition of method mtd"
  end
  def mtd
    puts "Second definition of method mtd"
  end
end


OR.new.mtd
```

Methods Overriding

```
class A
  def a
    puts 'In class A'
  end
end
class B < A
  def a
    puts 'In class B'
  end
end
b = B.new
b.a
```

Methods Overloading

```
class Rectangle
  def initialize(*args)
    if args.size < 2  || args.size > 3
      puts 'This method takes either 2 or 3 arguments'
    else
      if args.size == 2
        puts 'Two arguments'
      else
        puts 'Three arguments'
      end
    end
  end
end
Rectangle.new([10, 23], 4, 10)
Rectangle.new([10, 23], [14, 13])
```

Class Variables

```
class Student < Person

@@student_no = 0

    def initialize(name, age, job, branch)
        super(name, age, job)
        @branch = branch
    end

    def increment
      @@ student_no += 1
    end

end
```

Class Methods

```
class Example
    def instance_method # instance method
    end
    def self.class_method # class method
    end
end
```

Method Missing

```
class Dummy

  def method_missing(m, *args, &block)

    puts "There's no method called #{m} here
  -- please try again."

  end

end

Dummy.new.anything
```

Access control

```
class MyClass
    def method1 # default is 'public'
      #...
    end
    protected # subsequent methods will be 'protected'
    def method2 # will be 'protected'
      #...
    end
    private # subsequent methods will be 'private'
    def method3 # will be 'private'
      #...
    end
    public # subsequent methods will be 'public'
    def method4 # and this will be 'public'
      #...
    end
end
```

Access control

```
# by default, the instance variables are private to expose
  them you need setters and getters
class Song
  def initialize(name, artist)
    @name      = name
    @artist    = artist
  end
  def name
    @name
  end
  def name=(name)
    @name = name
  end
end
```

Modules/Mixins

```
# Mixins

# you can mix in more than one module in a class

class D

  include Stuff     # refers to the loaded module

  puts Stuff.m(4)

end
```

Thank you!